

# 校园时间管家 CampusTime

技术文档（AI 创新应用挑战赛 · 个人赛提交版）

AI 协同开发 · 细粒度 Git 提交 · Propose/Execute 确定性架构

|      |                                                                                                                          |
|------|--------------------------------------------------------------------------------------------------------------------------|
| 项目   | CampusTime / 校园时间管家 Agent                                                                                                |
| 定位   | 校园 AI 服务助手（学生生活端 / 驿站商户侧 / 管理控制台）                                                                                        |
| 技术栈  | Python + FastAPI + 原生 HTML/JS + 自写轻量 ReAct Agent + DeepSeek                                                              |
| 仓库   | <a href="https://github.com/zhoyuiliang0606/campus-time-agent">github.com/zhoyuiliang0606/campus-time-agent</a> (Public) |
| 文档版本 | 2026-09-25 · 提交历史 115 个细粒度 commit                                                                                        |
| 页数   | ≤ 30 页（本版约 22 页）                                                                                                         |

## 摘要

校园时间管家是一个把分散的校园信息（课表、待办、快递外卖、知识问答）收口到同一个 AI 助手的项目。其核心设计是 Propose / Execute 分离：Agent 只产出「提案」（read-only 工具），学生没在确认弹窗上点确认之前，系统一个字节都不写库；弹窗的显示由系统盖章的 kind 标记驱动，而非模型自觉。

本文档说明系统架构、确定性意图路由、本轮迭代（待办三类口语缺口修复）以及 AI 协同开发流程。全部代码以细粒度 commit 提交，开发对话以 Prompt 快照形式随代码入库，可逐提交还原协同过程。

## 提交清单对照

| 比赛要求                  | 本项目落地情况                              |
|-----------------------|--------------------------------------|
| Public 仓库 + 细粒度提交历史   | GitHub Public，115 个独立 commit，禁止一次性打包 |
| AI 对话历史 Prompt 快照     | ai-prompts/ 目录，每会话一文件，随代码提交          |
| README 复现指南           | 仓库根 README.md（起步 / 运行 / 测试 / 演示）     |
| 5 – 8 分钟演示视频（含 AI 协同） | docs/演示视频分镜.md（逐镜 + 解说词，用户本地录屏）      |
| 技术文档 PDF ≤30 页        | 本文件                                  |

1. 项目背景与定位

学生在校园里要同时盯课表、待办、取件、各类咨询，信息散落在不同系统。校园时间管家用「一个助手」把这些收口，分三层：

- 学生生活端（旗舰）：课表时间规划 + 校园问答。学生说一句话（如「把游泳排到周四 19:00」），系统算出提案、弹确认条，点确认才写进日程。
- 驿站 / 商户侧：监听快递外卖台账变动，主动推送取件提醒与滞留预警，用自定义人格智能回复咨询。
- 管理控制台：零代码维护知识库、配置客服人格。

三层共享同一个 Agent 引擎（意图路由 + 协调执行）+ 知识库 + 大模型，避免重复实现。

2. 技术栈与运行环境

| 层     | 选型                 | 说明                               |
|-------|--------------------|----------------------------------|
| 后端    | Python + FastAPI   | 把函数变成网页可调用接口；uvicorn 启动          |
| 前端    | 原生 HTML / JS       | 无需打包工具，确认弹窗走内嵌 /confirm 页面       |
| 大模型   | DeepSeek API       | Key 走环境变量 /.env，不写进代码            |
| Agent | 自写轻量 ReAct 循环      | 不套重框架，确定性分支优先于模型                 |
| 存储    | JSON 文件（app/data）  | 由 CAMPUSTIME_DATA_DIR 指向，测试用临时副本 |
| 测试    | FastAPI TestClient | pytest 风格自写 Checker，无需额外依赖       |

运行（复现摘要）

```
pip install -r requirements.txt
cp .env.example .env # ■■■ DEEPSEEK_API_KEY
uvicorn app.main:app --reload
# ■■■■■ http://127.0.0.1:8000
python tests/run_all.py # ■■■■■ 905/905
```

3. 系统总体架构

一次「学生说话 → 出提案 → 确认写库」的请求流：

```
■■■■■
■■■> /api/chat
■■■ router.route() ■■■■■■■■■■
■■■ ■■■■■■(3/3a/3b/3c/3d/3c-ter) ■■■■■■■■■■
■ ■■ is_confirmation+peek_pending ■■■■■■■■→ ■■■/■■■
■ ■■ wants_clear_timetable ■■■■■■+■■■
■ ■■ wants_add_todo ■■■■■■+■■■
■ ■■ wants_list_todo ■■■■■■■■
■ ■■ missing_thing_of ■■■■ X■■■■■■■
■■■ ■■■■■ → ■■(ReAct) ■ read-only ■■■■■■
■■■ = options[]■■■■■ kind ■■
■■■> ■■ renderOptions ■ kind ■■■■■
■■■> ■■■■■ → /api/... ■■■■■
```

关键点：写库的唯一入口是系统侧的确认动作，模型无论走哪条路都碰不到写入接口。这是「学生没点头就宣布已经删了/加了」这一类事故的硬防线。

## 4. 核心设计：Propose / Execute 分离

### 为什么

早期用纯模型直接回「已经帮你排好啦」，学生刷新日程却空空如也——模型没有写库能力，只是在文字里「表演」写库。把「说要做」和「真做了」混在一起，学生无法区分，是最伤信任的故障。

### 怎么做

- Agent 只调用 read-only 工具（查空档、列待办、读文件）。任何会改变状态的操作，都先变成一张「提案」返回，绝不在对话轮里直接落库。
- 提案由系统盖章 kind（如 `todo_add` / `timetable_clear` / `todo_mode` / `todo_slots`）。前端只按 kind 渲染对应弹窗——显示与否由系统决定，不由模型自报。
- 学生点弹窗上的【确认】后，才由系统真正执行写库；点【取消】则一切不变。

这正是「给 Agent 的提案做标记、前端识别后渲染」的落地形态；标记叫 kind，且必须由系统盖，不能由模型盖（否则「显不显示弹窗」的决定权又回到模型手里，正是「经常不弹弹窗」的成因）。

## 5. 确定性意图路由 (/api/chat)

为减少「模型自由发挥」导致的不可控，能在代码里定死的意图都收归确定性分支，模型只处理真正开放的问答。

| 分支    | 触发                          | 系统行为               | 写库?    |
|-------|-----------------------------|--------------------|--------|
| 确认执行  | is_confirmation + 有 pending | 按暂存提案写/删库          | 是（确认后） |
| 清空课表  | wants_clear_timetable       | 出清空提案+弹窗           | 否（等确认） |
| 加待办   | wants_add_todo              | 算时间+出确认条           | 否（等确认） |
| 读待办   | wants_list_todo             | list_todos() 真查并念出 | 否（纯读）  |
| 看不见 X | missing_thing_of            | 真查库核实，在/不在如实说      | 否      |

「不弹弹窗」的根因通常是：一句话没被任何确定性分支接住、掉进模型，模型只回纯文本（无 options/kind），弹窗自然不来。因此扩大确定性分支的覆盖面是修复弹窗问题的主战场。

## 6. 本轮迭代：待办三类口语缺口修复

### 问题

学生用口语说待办时，三种说法会「不弹弹窗 / 掉模型编瞎话」：

- 「我要加代办」——只有加的意图、没给具体内容，掉 FAQ/模型，无弹窗。
- 「我的代办呢 / 看看我的代办」——纯读，却无确定性分支，模型编「你目前有 3 条待办」（其实没读库）。
- 「代办显示不出来」——整张列表看不见，掉模型。

### 研判

读码确认 /api/chat 已在模型前做意图分析、提案由系统盖 kind；「不弹窗」= 意图没被接住。三者分别属于「加意图漏识」「缺只读列表分支」「报障措辞未覆盖」三类，逐个用确定性分支兜底即可，不必动模型。

### 改动

| 文件         | 改动                                                  |
|------------|-----------------------------------------------------|
| planner.py | _ADD_TALK_RE 加 代办/待办，让 wants_add_todo 接住「我要加代办」     |
| planner.py | 新增 wants_list_todo() 识别「我的代办呢」等纯读意图                 |
| planner.py | _MISSING_RE 加 显示不出来/显示不出，让「健身显示不出来」走核实分支            |
| main.py    | 在 3c-ter 前插入 3d 只读列表分支：list_todos() 真查、含星期显示，不写库不弹窗 |

### 验证

- 新增 tests/test\_07\_todo\_intent.py：19 项断言全绿（两开关识别、三类口语走系统分支、空列表分支）。
- 全量 tests/run\_all.py：通过 905 项，失败 0 项（共 7 批）。
- 关键行为：『我要加代办』→系统追问「要安排什么事」，未谎称已加；『我的代办呢』→真查库列出待办；『健身显示不出来』→核实「在的，日程里排着呢」。

## 7. 测试策略

测试不依赖 pytest，只用项目虚拟环境已有的 FastAPI TestClient + 自写 Checker，评委零额外安装即可跑。

- 每个测试文件在独立子进程 + 临时数据副本（sandbox）中运行，绝不污染真实演示数据。
- 护栏覆盖：加待办「出提案→确认→落库→刷新可见」全链路；删待办同理；清空课表；「看不见 X」核实；本轮三类口语。
- CI 式总账：run\_all.py 汇总每批「通过 X 项，失败 Y 项」，有失败退出码为 1。

```
python tests/run_all.py
# ■■■■■ 905 ■■■■ 0 ■■■■ 7 ■■■■■■ ■
```

## 8. AI 协同开发过程

本项目全程在人与 AI 协同下完成。为满足「细粒度提交 + Prompt 快照」要求，约定：

- 每做一步逻辑改动就单独 commit，绝不在最后一次性打包（当前 115 个细粒度 commit）。
- 每次开发会话写一份 Prompt 快照（ai-prompts/YYYY-MM-DD-<主题>.md），记录「用户 Prompt → AI 研判 → 改动 → 验证」。
- 快照文件随代码一起提交，评审把任意快照与其 commit 对照即可还原那一步协同过程。

示例（本会话）：用户问「有没有先分析用户的话再提交、再拉起弹窗」→ AI 读 /api/chat 确认确定性分支已在模型前做意图分析、提案由系统盖 kind → 定位三类待办口语缺口 → 用确定性分支兜底 → 19 项新测试 + 905 全量护栏 → 细粒度提交（0f4852a 及两份快照 eee76c9 / 041af38）。完整记录见 ai-prompts/2026-09-25-todo-gaps.md。

## 9. 复现指南（README 摘要）

- 环境：Python 3.10+，pip install -r requirements.txt。
- 密钥：cp .env.example .env 填入 DEEPSEEK\_API\_KEY（settings.json 已在 .gitignore 排除）。
- 启动：uvicorn app.main:app --reload，打开 http://127.0.0.1:8000。
- 自检：python tests/run\_all.py（临时数据副本，不碰演示数据）。

- 演示场景：说「帮我把今天的复习线性代数安排到 19:00-20:30」→看确认弹窗→点确认→日程出现；说「我的代办呢」→系统列出待办。

## 10. 局限与后续

- 确定性分支的覆盖面仍可扩大（如「游泳在哪天来着」走 schedule 模块时仍会反问，属同根病另一支）。
- 模型仅在开放问答时启用；高可靠操作（写/删）一律系统执行。
- 演示视频需本地录屏（沙箱无法录屏），逐镜脚本见 docs/演示视频分镜.md。

## 附录：本轮关键 commit

```
0f4852a ██████████/███
de6475b ██"███ x"████████████████████
0e2fa63 "██████x"██████████████████/██████
eee76c9 docs(ai-prompts): ███ AI ███ Prompt ██████████
041af38 docs(ai-prompts): 2026-09-25 ██████████ Prompt ███
```

— 文档结束 —